Aufgabenstellung: Java Zeit-APIs im Team entdecken

Jede Gruppe recherchiert eigenständig zu einem Themenkomplex der modernen Java-Zeit-API und bereitet eine **Mini-Präsentation (max. 10 Minuten)** für die anderen Gruppen vor.

Zeit bis 11 Uhr.

Themen & Gruppen

- **Gruppe 1:** LocalDateTime & DateTimeFormatter
- **Gruppe 2:** LocalDate & DateTimeFormatter
- **Gruppe 3:** LocalTime & DateTimeFormatter
- **Gruppe 4:** Period

Struktur für jede Gruppe

1. **Grundlagen**
    - Was repräsentiert euer Typ genau?
    - In welchem Package liegt er?
    - Was bedeutet das Wort *Local* in diesem Zusammenhang?
    - Vergleich zur alten API: Wie hätte man das früher mit java.util.Date oder Calendar gelöst?
2. **Immutability**
    - Was heißt „immutable“?
    - Warum ist das wichtig und welche Vorteile hat?
3. **Zentrale Methoden**
    - Wie erstellt man Objekte?
    - Wie liest man Werte aus?
    - Wie verändert man Werte (plus/minus, with…)?
    - Wie vergleicht man zwei Werte?
    - Gibt es besondere Hilfsmethoden?
4. **Codebeispiel**
    - Kurzes, funktionierendes Java-Programm (10–20 Zeilen)
    - Zeigt Erstellung, Manipulation, Formatierung und ggf. Ausgabe
    - Kleines Quiz

Gruppe 1: LocalDateTime & DateTimeFormatter

1. **Wie erstellt man ein LocalDateTime-Objekt**
    - **mit aktuellem Zeitpunkt (now()),**
    - **mit spezifischem Datum + Uhrzeit (of(year, month, day, hour, minute))?**
2. **Wie verbindet man ein LocalDate mit einem LocalTime zu einem LocalDateTime? (atTime())**
3. **Wie verschiebt man ein LocalDateTime um Tage, Monate oder Stunden? (plusDays, minusHours ...)**
4. **Wie extrahiert man Datum und Uhrzeit getrennt? (toLocalDate(), toLocalTime())**
5. **Wie formatiert man LocalDateTime mit DateTimeFormatter?**
    - **Unterschied zwischen ISO- und benutzerdefinierten Mustern**
    - **Warum ist DateTimeFormatter thread-sicher, SimpleDateFormat aber nicht?**
6. **Beispiel:**
    - **Erstelle einen Termin am 20.01.2027, 17:30 Uhr.**
    - **Verschiebe ihn um 30 Minuten.**
    - **Gib ihn im Format "dd.MM.yyyy HH:mm" aus.**
7. **Vergleich: Wie wäre das mit Date / Calendar / SimpleDateFormat gegangen?**

Gruppe 2: LocalDate & DateTimeFormatter

1. **Was repräsentiert LocalDate genau (nur Datum, ohne Zeit und Zeitzone)?**
2. **Wie erstellt man ein LocalDate:**
    - **Aktuelles Datum (now()),**
    - **Festes Datum (of(year, month, day)).**
3. **Wie verändert man ein Datum (plusDays, minusWeeks, withMonth)?**
4. **Wie vergleicht man zwei Daten (isBefore, isAfter, isEqual)?**
5. **Wie prüft man, ob ein Jahr ein Schaltjahr ist (isLeapYear())?**
6. **Formatierung mit DateTimeFormatter:**
    - **Formatierung mit Standard-ISO (ISO_DATE)**
    - **Benutzerdefiniertes Muster ("dd.MM.yyyy")**
    - **Achtung: Uhrzeit-Muster (HH:mm) führen zu Fehlern!**
7. **Parsing:**
    - **Wie kann man aus einem String wieder ein LocalDate machen? (LocalDate.parse(text, formatter))**
8. **Beispiel:**
    - **Erstelle dein Geburtsdatum.**
    - **Berechne den Wochentag deines Geburtstags im Jahr 2030.**
    - **Formatiere es als "EEEE, dd.MM.yyyy".**
9. **Vergleich:**
    - **Wie hätte man das früher mit Date oder Calendar gelöst?**
    - **Welche Vorteile bringt die neue API (Lesbarkeit, Sicherheit, einfache Rechenoperationen)?**

Gruppe 3: LocalTime & DateTimeFormatter

1. **Was repräsentiert LocalTime? (Nur Uhrzeit, ohne Datum, ohne Zeitzone)**
2. **Wie erstellt man ein LocalTime:**
    - **Aktuelle Zeit (now()),**
    - **Feste Uhrzeit (of(hour, minute, second)).**
3. **Wie verändert man Uhrzeiten (plusMinutes, minusHours, withSecond)?**
4. **Wie vergleicht man Zeiten (isBefore, isAfter)?**
5. **Wie liest man Teile aus (getHour, getMinute, getSecond)?**
6. **Formatierung mit DateTimeFormatter:**
    - **Standardformat: ISO_TIME**
    - **Eigenes Muster: "HH:mm:ss" oder "HH:mm"**
    - **Warum kann man hier keine Datumsbestandteile verwenden? (UnsupportedTemporalTypeException)**
7. **Parsing:**
    - **Wie kann man einen Text wie "15:45" in eine Zeit umwandeln? (LocalTime.parse(text, formatter))**
8. **Beispiel:**
    - **Prüfe, ob eine Zeit (z. B. 15:45) innerhalb der Geschäftszeit 08:00 – 18:00 Uhr liegt.**
    - **Falls nicht, berechne den nächsten Öffnungszeitpunkt.**
9. **Vergleich:**
    - **Früher: Zeitberechnung über Calendar umständlich und fehleranfällig.**
    - **Heute: klar, typensicher, immutable.**

Gruppe 4: Period

1. **Was ist eine Period und wofür wird sie verwendet? (Differenz zwischen zwei LocalDates)**
2. **Wie erstellt man Perioden:**
    - **Manuell (ofYears, ofMonths, ofDays),**
    - **Berechnet (Period.between(start, end)).**
3. **Wie liest man Werte aus (getYears, getMonths, getDays)?**
4. **Wie addiert/subtrahiert man Perioden (date.plus(period), date.minus(period))**
5. **Unterschied zu Duration (arbeitet mit Sekunden/Nanosekunden, für Zeiten).**
6. **Beispiel:**
    - **Berechne das Alter einer Person anhand ihres Geburtsdatums.**
    - **Gib das Ergebnis aus:**
      „Du bist X Jahre, Y Monate und Z Tage alt."
7. **Vergleich:**
    - **Früher: Millisekunden-Differenzen mit Date oder Calendar → unverständlich.**
    - **Neu: Semantisch klar und leicht zu lesen.**